

Multi-Declaration Z Constructs — Demo

txt2tex

May 20, 2026

Background

The constructs in this file all share a single grammatical shape: a Z paragraph that introduces two or more typed variables in one declaration list, with a conjunction predicate over their projections, optionally followed by a characteristic expression. Until the multi-decl parser fix, none of them parsed natively — users wrote them as raw LaTeX. They now compile to fuzz-clean output.

$[PlayerName, TournamentId, MatchId, VenueName]$

<i>Tournament</i>
<i>tournamentId</i> : <i>TournamentId</i>
<i>venue</i> : <i>VenueName</i>
<i>tier</i> : $\mathbb{N}$
<i>tier</i> ≤ 5

$PK(Tournament) = \{tournamentId\}$

<i>Player</i>
<i>name</i> : <i>PlayerName</i>
<i>ranking</i> : $\mathbb{N}$

$PK(Player) = \{name\}$

<i>Match</i>
<i>matchId</i> : <i>MatchId</i>
<i>tournament</i> : <i>TournamentId</i>
<i>winner</i> : <i>PlayerName</i>
<i>loser</i> : <i>PlayerName</i>

$PK(Match) = \{matchId, tournament\}$

1 . WYSIWYG line breaks for algebra operators

Long relational-algebra chains can be broken at any operator. Natural newlines work; explicit continuation also works. Display position wraps in array; the chain reads top-to-bottom instead of running off the page.

$Project\{winner, venue, tier\}(Tournament \otimes$
 $Rename_{tournament \rightarrow tournamentId}(Project\{tournament\}(Restrict_{winner='Ali'}(Match))))$

2 . Multi - decl comprehension with conjunction predicate

This is the canonical multi-typed relational-calculus form for any query that crosses two or more relations. Before the fix it required a raw LATEX block; now it parses natively.

$$\{ m : Match; t : Tournament; p : Player \mid m.tournament = t.tournamentId \wedge m.winner = p.name \wedge t.venue = 'CentreCourt' \bullet \langle winner == m.winner, venue == t.venue, tier == t.tier \rangle \}$$

3 . Multi - decl quantifier with conjunction predicate

‘ $\forall s : T; c : U \bullet predicate. \text{ body}$ ’ works the same way — nested Quantifier nodes covering the chained declarations.

$$\forall m : Match; t : Tournament \mid m.tournament = t.tournamentId \wedge t.tier \leq 2 \bullet m.winner \neq m.loser$$

4 . Multi - decl quantifier without characteristic expression

When the predicate alone suffices — no ‘.’ separator, no body expression — the comprehension defaults the characteristic tuple to the signature (Z RM §3.9).

$$\{ m : Match; t : Tournament \mid m.tournament = t.tournamentId \wedge t.tier \leq 2 \}$$

5 . Multi - decl mu (definite description)

‘mu’ parses with multi-decl signatures the same way. The current generator emits nested ‘ $(\mu \dots \mid (\mu \dots))$ ’ — *semantically equivalent but not Spivey - canonical. The same special - case treatment lambda just received is a follow - up for mu. Single - decl mu (shown here) emits cleanly.*

$$topFinal == (\mu t : Tournament \mid t.tier = 1 \bullet t.venue)$$

6 . Multi - decl lambda — Spivey - canonical form

Lambdas of more than one variable now emit as a single λ token with semicolon - joined SchemaText, matching ZRM 3.12 and the

$$roster == (\lambda m : Match; t : Tournament \mid m.tournament = t.tournamentId \bullet m.winner)$$

7 . Paren - wrap fix

Fuzz requires ‘ $(\lambda \dots \bullet \dots)$ ’ — *parentheses around every lambda expression, at any position. The generator previously only wrapped leveled predicates emitted bare ‘ λ ’, which fuzz rejected with Opening parenthesis expected at symbol λ . The fix : drop the ‘paren’*
 $(\lambda x : \mathbb{N} \bullet x + 1)$